

DineCompass:
An Evidence-Grounded Restaurant Recommendation System

5293 Final Project Report

Name	Uni
Quanjing He	qh2305
Songyan Cai	sc5756
Xiying Chen	xc2781

Abstract

DineCompass is an evidence-grounded restaurant recommendation system designed to help users make more informed dining decisions. Unlike traditional restaurant search platforms that mainly rely on ratings, distance, or manual review reading, DineCompass combines structured restaurant metadata, review-based evidence, multimodal image analysis, and explainable ranking into one personalized recommendation pipeline. Given a user’s ZIP code, preferred cuisine, party size, and budget range, the system retrieves nearby restaurant candidates through Google Places, extracts relevant review snippets using a lightweight retrieval-augmented generation approach, and applies Gemini-based language and vision analysis to summarize textual and visual dining signals. These signals are organized into structured restaurant dossiers and ranked using an interpretable rule-based scoring module. The final web interface returns Top-3 recommendation cards with restaurant information, review evidence, image-based observations, recommended dishes, reservation or map links, and score breakdowns. Experimental tests and user feedback show that DineCompass provides more transparent, grounded, and actionable recommendations than general LLM outputs or standard restaurant search tools. Overall, this project demonstrates how LLMs, RAG-style review retrieval, VLM analysis, and explainable ranking can be integrated into a practical restaurant recommendation system that better reflects real dining preferences and decision-making needs.

1 Introduction

1.1 Project Background

Choosing a restaurant is often more complicated than simply sorting by rating, distance, or price. A restaurant with a high rating may still not be the best choice for a large group, a quiet dinner, a casual meal, or a specific food craving. In real dining decisions, people usually consider a mix of practical and subjective factors, such as recommended dishes, service quality, wait time, value for money, atmosphere, seating comfort, and how the food or space looks in photos.

This makes restaurant recommendations a good setting for large language models and multimodal analysis. Much of the useful information already exists online, but it is scattered across restaurant metadata, reviews, ratings, and customer photos. Our original proposal described this as a dining intelligence task: the goal is to help users decide what to eat and which restaurant to choose by collecting fragmented restaurant information and turning it into a more reliable restaurant profile.

We name our system DineCompass because the project is meant to guide users through nearby dining choices. The word “Compass” reflects the role of the system: it starts from the user’s location, searches nearby restaurants, and helps the user move toward options that better match their current needs. Instead of acting like a normal map search tool, DineCompass adds review understanding, image-based observations, and explainable ranking to support the final recommendation.

1.2 Project Goal

The goal of DineCompass is to build a personalized restaurant recommendation website. Given a user’s ZIP code, preferred cuisine, party size, and budget range, the system returns the Top-3 restaurants that best match the user’s current dining context.

The output is designed to be more informative than a standard restaurant list. For each recommended restaurant, DineCompass provides restaurant metadata, a short recommendation explanation, review-based evidence, image-based observations when available, and a score breakdown. This helps users understand not only which restaurants are recommended, but also why those restaurants are suitable for their request.

In the current implementation, DineCompass uses Google Places, Gemini-based text and image analysis, and an explainable rule-based ranker to generate evidence-supported Top-3 recommendations.

For the final prototype, we prioritize a stable end-to-end workflow. The implemented system uses Google Places as the main restaurant data source, Gemini for review and image analysis, and a rule-based ranking module for interpretable recommendation. Additional sources such as Yelp and Reddit, as well as more advanced corrective RAG, are left for future extensions.

1.3 Main Contributions

The first contribution of DineCompass is that it combines several types of restaurant information in one pipeline. Instead of relying only on ratings or distance, the system brings together Google Places metadata, review snippets, LLM-generated review analysis, restaurant photos, and VLM-based visual descriptions.

The second contribution is the use of a lightweight RAG-style review evidence layer. Reviews are first converted into evidence chunks, and the system retrieves the snippets that best match the user’s request before asking the LLM to summarize them. This makes the recommendation more grounded in actual review evidence instead of relying only on a general model response.

The third contribution is the explainable ranking design. DineCompass does not simply ask an LLM to choose the best restaurant. Instead, it calculates an interpretable score breakdown based on multiple ranking factors and combines these components into a total score. The ranker sorts restaurants by this total score and returns the strongest candidates.

2 Related Work and Literature Review

2.1 Restaurant Recommendation Systems

Restaurant recommendation is closely related to location-based and preference-aware recommendation. Earlier work on location-based recommender systems shows that venue recommendation should consider not only geographic distance, but also user preferences and social opinions. Bao et al.^[1] describe a location-based and preference-aware recommender system that recommends venues such as restaurants within a geospatial range while considering personal preferences and local opinions.

DineCompass follows a similar motivation but implements it in a smaller course-project setting. Unlike many traditional recommender systems that rely on historical user profiles or large-scale behavioral data, DineCompass is designed for a cold-start setting. We do not train a full recommender model from user histories. Instead, the system takes direct user input, retrieves nearby restaurant candidates from Google Places, and uses reviews and photos to add more context to each candidate. This design is more realistic for our prototype because we do not have long-term user behavior data, but we can still build useful recommendations from public restaurant information and user-provided preferences.

2.2 LLMs for Review Understanding

Restaurant reviews contain many useful signals, but they are written in free-form natural language. A review may mention that the food is good, the service is slow, the space is crowded, or the portions are worth the price. These details matter for dining decisions, but they are not usually stored as clean structured fields.

This task is related to aspect-based opinion mining, where review text is analyzed by specific aspects such as food quality, service, price, and ambience rather than only by overall sentiment ^[2].

DineCompass uses an LLM to turn retrieved review evidence into structured restaurant insights. The system schema includes review analysis fields such as signature dishes, service, value, wait impression, vibe, pros, cons, and supporting evidence. It also defines fallback values such as “Unknown” when reliable information is unavailable. This makes the output more stable because the system can still return a usable dossier even when some review information is missing or incomplete.

2.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation is useful for this project because restaurant explanations should be grounded in actual evidence. RAG combines generation with retrieved information, so the model can produce answers based on external evidence rather than only relying on its internal knowledge. Lewis et al. ^[3] describe RAG as a method that combines parametric model memory with non-parametric retrieved memory for knowledge-intensive generation tasks.

In DineCompass, the same idea is applied to restaurant reviews. Instead of asking the LLM to generate restaurant explanations from general knowledge, the system first retrieves review evidence and then asks the model to summarize only the selected evidence. This helps make the recommendation more trustworthy because the final explanation can be connected back to actual customer reviews.

Compared with a full vector-database RAG system, our keyword-overlap retrieval is simpler and easier to debug, but it may miss semantically relevant reviews that use different wording. This trade-off is acceptable for the prototype but motivates embedding-based retrieval as future work.

2.4 Vision-Language Models for Visual Preference Matching

Photos play an important role in restaurant choice because some dining features are easier to observe visually than to infer from ratings or short metadata. For example, a user may want to know whether a restaurant looks crowded, casual, upscale, spacious, or suitable for a group. These details may not be clearly captured in structured restaurant fields, but they can often be observed from customer photos.

Compared with text-only review analysis, vision-language models add a complementary perspective because visual features such as seating density, lighting, food presentation, and atmosphere can be interpreted from images. Vision-language models are designed to connect visual inputs with language-based understanding, which makes them suitable for analyzing restaurant photos and converting visible cues into text descriptions ^[9].

In DineCompass, VLM analysis is used as an additional evidence layer rather than as the main decision maker. The goal is not to let the image model directly choose the best restaurant, but to add visual context that can support the final restaurant dossier and ranking.

2.5 Explainable Recommendation

Explainability matters in restaurant recommendation because users may not trust a result if they only see a name and a score. Explainable recommendation research argues that recommendation systems should help users understand why an item is recommended. Zhang and Chen^[4] describe explainable recommendations as a way to improve transparency, trust, persuasiveness, user satisfaction, and system debugging.

DineCompass follows this idea through both evidence and scoring. Review snippets show what textual evidence supports the recommendation, while image analysis adds visual cues when photos are available. The score breakdown shows how each restaurant performs across several interpretable ranking factors, making the recommendation process more transparent to users.

2.6 Open-Source Project Inspirations

In addition to academic work on location-based recommendation, retrieval-augmented generation, multimodal analysis, and explainable recommendation, DineCompass was also informed by two open-source agent projects. The Best-Restaurants-Route-Planner-Agent project ^[5] motivated our use of external map-based tools for restaurant discovery. However, DineCompass differs from this project because it focuses on evidence-backed restaurant recommendation rather than route optimization.

The ecommerce-competitor-analyzer project ^[6] provided a useful design reference for collecting information from external sources, analyzing textual and visual materials, and generating structured outputs. DineCompass adapts this general idea to the restaurant domain. Instead of producing a competitor analysis report, the system generates restaurant dossiers that summarize review evidence, visual cues, and ranking reasons for dining decisions.

3 System Overview

3.1 User Interface

DineCompass provides a web interface where users enter their dining preferences. The main user-facing inputs are ZIP code, cuisine craving, party size, and budget range. For the course demo, the interface also includes fields for a Google Maps API key and a Gemini API key. These keys are used by the backend to call Google Places and Gemini services during the live search process.

This API-key input design is a prototype-level implementation choice. During live testing, shared API keys may encounter quota and rate-limit constraints. Allowing users to provide their own keys reduces dependence on a single team-managed key and avoids exposing private credentials in a public or semi-public deployment environment. In a production system, API credentials should be managed securely on the server side through environment

variables or a secret-management service, together with access control, quota monitoring, caching, and billing protection.

The backend request schema validates the main input fields. The required fields include zipCode, cuisine, partySize, and budget. The schema restricts party size to a valid range and requires the budget to be one of low, medium, high, or luxury. It also supports optional preference fields such as value, service, wait time, vibe, signature dishes, group suitability, portion size, quietness, casualness, and upscale preference.

After the user submits the form, the system displays Top-3 recommendation cards. Each card includes restaurant information, generated dining insights, review evidence, image analysis when available, and a score breakdown. The interface is designed to help users compare restaurants quickly without reading many individual reviews themselves.

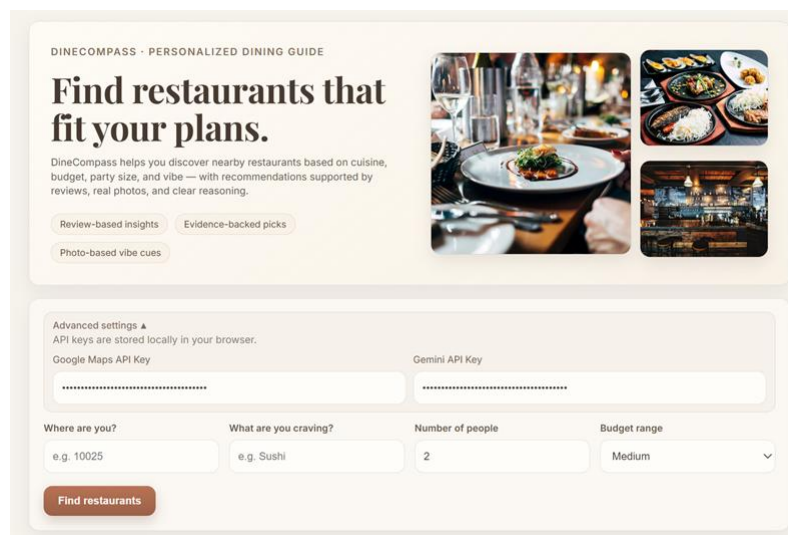
The image shows a web interface for 'DINECOMPASS - PERSONALIZED DINING GUIDE'. The main heading is 'Find restaurants that fit your plans.' Below this, a subtext explains that the service helps discover nearby restaurants based on cuisine, budget, party size, and vibe, supported by reviews, photos, and reasoning. There are three buttons: 'Review-based insights', 'Evidence-backed picks', and 'Photo-based vibe cues'. To the right of the text are three images: a restaurant interior, a plate of food, and a restaurant exterior. Below the main content is an 'Advanced settings' section with a note that API keys are stored locally. It contains two input fields for 'Google Maps API Key' and 'Gemini API Key'. Below these are four input fields: 'Where are you?' (with example 'e.g. 10025'), 'What are you craving?' (with example 'e.g. Sushi'), 'Number of people' (with value '2'), and 'Budget range' (with a dropdown menu showing 'Medium'). A 'Find restaurants' button is at the bottom left of the form.

Figure 1. Web Interface of DineCompass

The DineCompass web interface collects user dining preferences, including ZIP code, cuisine, party size, and budget. For the course demo, users can also provide their own Google Maps and Gemini API keys to support stable live testing. The system returns evidence-backed recommendation cards with review insights, visual cues, and score breakdowns.

3.2 Backend Architecture

The backend of DineCompass is built with FastAPI and organized as a modular pipeline. Instead of placing all logic in one large script, the system separates the recommendation process into several components: frontend interaction, request orchestration, data retrieval, review evidence processing, multimodal analysis, dossier generation, ranking, and final result rendering.

At a high level, the web interface collects the user's dining preferences and sends them to the FastAPI search route. The search route works as the main coordinator of the backend. It validates the request, calls the necessary service modules in order, and returns structured recommendation results to the frontend. The request and response formats are defined

through schemas, while prompt files provide task-specific instructions for review analysis, image analysis, and dossier generation.

The backend is organized into modular service components rather than one large script. These components handle location processing, restaurant retrieval, review evidence processing, LLM-based text analysis, VLM-based image analysis, dossier generation, and ranking. This modular design makes the system easier to debug, test, and extend because each service has a clearly defined responsibility.

Figure 2 summarizes this architecture. The diagram shows how the frontend, FastAPI backend, service modules, prompts, schemas, and output cards are connected. It also separates the main runtime pipeline from development and testing files, which helps clarify which parts are used during recommendation generation and which parts support setup or debugging.

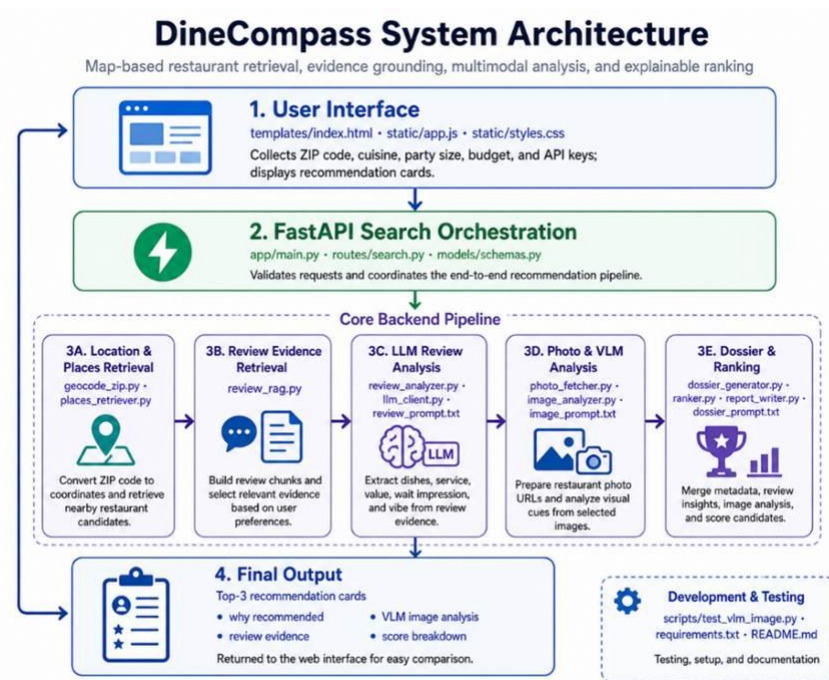


Figure 2. System Architecture of DineCompass

DineCompass is organized as a modular FastAPI-based pipeline. The web interface collects user preferences, the backend coordinates restaurant retrieval and evidence processing, LLM and VLM modules analyze reviews and images, and the ranking module returns Top-3 recommendation cards with supporting evidence and score breakdowns.

3.3 End-to-End Pipeline

During a user search, DineCompass follows a fixed end-to-end workflow. The pipeline can be summarized as:

User Input → ZIP Code Geocoding → Restaurant Retrieval → Review Evidence Construction → LLM Review Analysis → Photo Preparation → Preliminary Ranking → VLM Analysis for Top Candidates → Final Ranking → Recommendation Cards

First, the user enters a ZIP code, cuisine preference, party size, and budget range. The system converts the ZIP code into latitude and longitude, then uses the location and cuisine keyword to retrieve nearby restaurant candidates. For each candidate, the system collects structured place information such as name, address, rating, number of ratings, price level, reviews, photos, and map-related links.

Next, the system processes the review and photo data. Reviews are converted into evidence chunks, and the most relevant snippets are selected based on the user's request. These snippets are passed to the LLM review analyzer, which extracts structured signals such as signature dishes, service quality, value, wait impression, and vibe. In parallel, restaurant photo references are converted into usable photo URLs.

After the text-based analysis is complete, DineCompass builds an initial dossier for each restaurant and performs a preliminary ranking. To keep the system efficient and avoid unnecessary multimodal API calls, VLM image analysis is applied only to the strongest preliminary candidates. The final dossiers are then rebuilt with visual evidence, ranked again, and returned to the frontend as Top-3 recommendation cards.

This staged workflow makes the recommendation process more controllable and interpretable than a single-step chatbot response. Each component has a clearly defined role: retrieval identifies candidate restaurants, review evidence grounds the textual analysis, VLM analysis adds visual context, dossier generation organizes heterogeneous information into structured profiles, and rule-based ranking produces a transparent final ordering.

4 Data Collection and Preprocessing

4.1 User Input Data

DineCompass begins with user-provided search preferences. The core input fields are ZIP code, cuisine, party size, and budget. These fields define the restaurant search space and the ranking criteria.

The ZIP code determines the geographic search location. Cuisine is used as a keyword for restaurant retrieval and as a matching signal in ranking. Party size helps evaluate whether a restaurant seems suitable for the user's group. The budget is mapped to a target price level, which is compared with each restaurant's price level during scoring.

The system also supports optional preference fields. These include value, service, wait time, vibe, signature dishes, group suitability, portion size, quietness, casualness, and upscale preference. Although the current interface focuses mainly on the required fields, these optional fields make the system easier to extend.

4.2 Google Maps / Places Data

DineCompass uses Google Places as the main restaurant data source [7]. After the ZIP code is converted into coordinates, the system sends a Nearby Search request using latitude,

longitude, cuisine keyword, and restaurant type. The search radius is set to 5000 meters, and the system limits the number of processed restaurant candidates to 10.

For each candidate, the system retrieves richer restaurant information through Place Details. The restaurant schema stores fields such as place ID, name, formatted address, rating, user rating count, price level, types, photos, reviews, reservation information, maps link, and geographic location. These fields form the factual base for the later review analysis, image analysis, dossier generation, and ranking.

4.3 Review Evidence Construction

Review preprocessing turns raw Google review text into structured evidence items. Each evidence item stores the review text, rating, author name, relative time description, source label, and matched terms. Empty review text is skipped so that the later analysis is based only on usable customer comments.

After the evidence items are created, the system prepares a smaller evidence set for each restaurant. In the search route, up to three review snippets are selected for each candidate restaurant before review analysis. This keeps the input to the LLM concise and makes the final recommendation easier to trace back to specific review examples.

This preprocessing step does not try to make the final recommendation by itself. Its role is to clean and organize review data into a format that can be used by the retrieval and review-analysis modules.

4.4 Image Data Preparation

Restaurant image data comes from Google Places photo references. For each restaurant, the photo fetcher converts available photo references into Google Place Photo URLs. The photo URL builder uses a maximum width of 900 pixels, and the photo URL list builder supports a limit of three photos per restaurant.

The system does not assume that every restaurant has usable photos. If photo references are missing or cannot be converted into valid image URLs, the restaurant can still be processed using metadata and review evidence. This fallback behavior helps keep the application stable even when image data is incomplete.

After image URLs are prepared, they can be passed to the VLM module for selected candidate restaurants. This step separates image collection from image interpretation, making the data preparation process clearer and easier to debug.

5 Methodology

5.1 Hybrid Recommendation Framework

This project adopts a hybrid recommendation framework for personalized restaurant search. Instead of relying only on traditional signals such as rating, price, or distance, the system combines structured restaurant metadata, review-based textual evidence, visual information from restaurant photos, and an explainable ranking mechanism. This design is motivated by the fact that real dining decisions often depend on both objective attributes and softer preferences, such as service quality, atmosphere, food presentation, wait experience, and group suitability.

The framework combines retrieval, language understanding, visual understanding, and transparent ranking. Structured metadata provides factual restaurant information, review evidence grounds the text analysis, image analysis adds visual context, and the rule-based ranker turns these signals into an interpretable recommendation score.

A key design choice is that the final ranking is not directly determined by the LLM. Instead, the LLM and VLM modules are used mainly for understanding and summarizing unstructured evidence, while the final recommendation order is produced by a rule-based scoring system. This allows the system to provide both personalized results and interpretable score breakdowns. Therefore, the framework can be viewed as a combination of retrieval, language understanding, visual understanding, and transparent ranking, rather than a black-box recommender.

5.2 Retrieval-Augmented Review Understanding

To make the recommendation rationale more grounded, the system uses a lightweight retrieval-augmented review understanding module. Instead of sending all available Google reviews directly to the LLM, the system first converts each review into an evidence chunk and retrieves the snippets that are most relevant to the user's request. This follows the general RAG idea: the model should analyze selected evidence rather than generate recommendations from unsupported assumptions.

In this project, the retrieval step is implemented as lexical keyword matching rather than embedding-based semantic search. The system builds query terms from the user request, including cuisine, budget, party size, value preference, service preference, wait preference, vibe, group suitability, portion size, signature dishes, and optional style indicators such as quiet, casual, or upscale. Each restaurant review is tokenized, and the system counts how many query terms overlap with the review text. Reviews with more matched terms are ranked higher, and the top three review snippets are selected as review evidence for that restaurant. If no matching terms are found, the system falls back to the first available reviews.

This design has two advantages. First, it reduces noise by focusing the LLM on reviews that are more connected to the user's dining preferences. Second, it improves interpretability

because the selected reviews are shown later as “Review RAG Evidence” in the result card. Therefore, the recommendation is not only based on structured metadata such as ratings and price level, but also supported by concrete customer review evidence. The project README explicitly describes this module as a lightweight review RAG approach that retrieves relevant snippets using keyword overlap, without a vector database.

5.3 LLM-based Structured Signal Extraction

After the relevant review evidence is retrieved, the system uses Gemini to convert unstructured customer reviews into structured restaurant-level signals [8]. Raw reviews are difficult to use directly because they may contain mixed opinions about food, service, value, waiting time, and atmosphere. The LLM-based review analyzer addresses this problem by extracting concise fields from the selected review snippets, including `signature_dishes`, `service`, `value`, `wait_impression`, `vibe`, `pros`, `cons`, and `supporting_evidence`.

The review prompt is designed to keep the LLM output grounded. It explicitly instructs the model to analyze only the provided review evidence, avoid unsupported facts, and return a strict JSON object with the required keys. If the evidence is weak, the prompt asks the model to mark the corresponding field as “Unknown” rather than inventing information. This constraint helps reduce hallucination and makes the extracted signals more reliable for downstream recommendation and ranking.

This module also includes fallback handling. If no review evidence is available, the system returns an “Unknown” review analysis with a note that no evidence was available. If the Gemini call fails, the system still returns a valid review analysis object and uses a rule-based fallback to extract explicit dish mentions such as sushi, ramen, noodles, pasta, tacos, or pizza from the retrieved reviews. Therefore, the LLM is used as a structured interpretation layer, while the system remains functional even when model output is unavailable.

5.4 Vision-Language Model for Visual Preference Matching

Text reviews do not fully capture the visual aspects of a dining experience, such as restaurant ambience, seating space, food presentation, and whether the environment looks suitable for a small or large group. To address this limitation, the system uses a Vision-Language Model (VLM) to analyze restaurant photos and convert visual information into structured recommendation signals.

In this project, image analysis is performed with Gemini 2.5 Flash through a multimodal Gemini call. The VLM prompt instructs the model to analyze only visible evidence from the restaurant images and avoid using review text or outside knowledge. The output must be JSON strict with fields including `visual_vibe`, `space_impression`, `food_visual_cues`, `group_suitability`, `visual_confidence`, and `image_evidence_summary`. If the image is unclear or does not provide enough information, the model is instructed to use “unknown” instead of inventing details.

For efficiency, the system does not run VLM analysis on every candidate restaurant. It first builds initial dossiers with default image analysis, ranks the candidates using rule-based scoring, and then applies VLM analysis only to the preliminary Top-3 restaurants. For each of these restaurants, only the first photo URL is passed to the VLM. After the image analysis is completed, the system rebuilds the dossiers with the visual signals and performs the final ranking.

This design allows the recommender to incorporate visual preference matching while controlling API cost and latency. The VLM does not directly choose the final recommendations; instead, it provides additional visual evidence that supports later dossier generation and ranking. As a result, the system can consider not only what customers say in reviews, but also what the restaurant looks like in available photos.

5.5 Dossier-based Recommendation Representation

After the review and image analysis steps, the system builds a structured restaurant dossier for each candidate restaurant. The dossier serves as an intermediate representation that combines multiple information sources into one consistent format. Instead of passing raw Google Places data, review snippets, and image analysis separately to the final ranking and frontend display, the system organizes them into a unified restaurant profile.

The dossier generation module takes five main inputs: the restaurant metadata from Google Places, the LLM-based review analysis, the user request, the VLM-based image analysis, and the available photo URLs. These inputs are inserted into a Gemini prompt, which asks the model to generate concise, evidence-grounded fields such as signature dishes, service, value, wait impression, vibe, summary, and why recommended. The prompt explicitly requires the model to ground all claims in the provided input, avoid unsupported facts, use “Unknown” when uncertain, and return strict valid JSON.

The final dossier includes both original place information and generated recommendation text. Its fields include restaurant name, rating, price level, address, restaurant types, review evidence, image analysis, photo URLs, reservation or Google Maps links, summary, why recommended, and recommendation confidence. This structure makes the downstream ranking and result rendering more stable because every restaurant is represented with the same schema.

The dossier module also includes fallback behavior. If Gemini fails to return a valid response, the system still creates a dossier using available metadata, review analysis, image analysis, and a fallback summary or recommendation reason. The recommendation confidence is also assigned based on the amount of review evidence and the confidence level of the image analysis. In this way, the dossier acts as a reliable bridge between raw data collection, LLM/VLM interpretation, and final explainable ranking.

5.6 Explainable Rule-based Ranking

The final recommendation order is generated by an explainable rule-based ranking module rather than directly by the LLM. This design makes the recommendation process more transparent: the LLM and VLM are used to extract and summarize evidence, while the ranking module assigns explicit scores to each restaurant based on predefined criteria.

For each restaurant dossier, the ranker calculates a score breakdown with eight components: `cuisine_match`, `budget_match`, `rating`, `review_value_match`, `vibe_fit`, `visual_vibe_fit`, `evidence_quality`, and `wait_penalty`. The total score is computed as the sum of these components, and restaurants are sorted in descending order by total score. This score breakdown is also returned in the API response, so the frontend can show users why a restaurant was ranked highly.

The scoring rules combine structured metadata and extracted signals. `cuisine_match` checks whether the requested cuisine appears in the restaurant types. `budget_match` compares the user’s budget level with the restaurant’s Google Places price level, where low, medium, high, and luxury are mapped to price levels 1 to 4. `rating` converts the Google rating into a score out of 20. `review_value_match` uses the extracted value field from the dossier to reward positive value descriptions such as “great,” “excellent,” or “good,” and penalize negative descriptions such as “poor” or “overpriced.”

The ranking also considers user context and evidence quality. `vibe_fit` checks whether the text-based vibe is suitable for the party size, such as spacious or group-friendly for larger parties, and cozy, romantic, or casual for smaller parties. `visual_vibe_fit` applies a similar idea to the VLM image analysis, using visual terms related to group suitability or intimate dining. `evidence_quality` gives more credit to restaurants with available review evidence, while `wait_penalty` subtracts points when the extracted wait impression suggests long waits or busy conditions.

Overall, this ranking method keeps the final recommendation interpretable and controllable. Instead of treating the LLM as a black-box decision maker, the system uses LLM/VLM outputs as evidence-based features and then applies transparent scoring rules to produce the final Top-3 recommendations.

6 Experimental Setup

6.1 Implementation Environment

We implemented the experiment as a local web-based restaurant recommendation system using Python and FastAPI. The system was designed as an LLM + RAG + VLM demo that takes user dining preferences and returns Top-3 restaurant recommendations with supporting evidence.

The backend was organized into separate modules for ZIP code geocoding, Google Places retrieval, review evidence retrieval, Gemini review analysis, photo fetching, Gemini image analysis, dossier generation, and ranking. This modular structure made the system easier to debug, test, and extend.

The main backend app was built in `app/main.py`. It connected the FastAPI app, frontend templates, static files, and search router. We also added a GET `/health` endpoint to check the service status. The main search function was handled by the POST `/api/search` endpoint, which received the Google Maps API key, Gemini API key, ZIP code, cuisine type, party size, budget level, and optional dining preferences.

The main tools and libraries included FastAPI, uvicorn, Jinja2, httpx, python-dotenv, and Pydantic. We ran the system locally with `uvicorn app.main:app --reload --port 8000` and opened `http://localhost:8000` to use the frontend.

The system used the Google Maps / Google Places API to geocode the ZIP code and retrieve nearby restaurant candidates. It also used the Gemini API for review analysis and image analysis. We used Pydantic to validate the request schema, including ZIP code format, party size range, and budget level.

Because DineCompass uses pretrained Gemini models and a manually designed rule-based ranker, no custom model training was performed in this project. Instead, optimization focused on prompt design, evidence selection, fallback handling, and API-call efficiency. For example, the system retrieves only the top three review evidence snippets for each restaurant and applies VLM image analysis only to the preliminary Top-3 candidates. This reduces unnecessary API calls while keeping the final recommendations evidence-grounded and interpretable.

6.2 Test Query Design

We tested the system with three different queries to check whether it could adjust recommendations based on user needs.

For Test 1, we used ZIP code 10018, cuisine sushi, party size 2, and budget medium. The Top-3 results focused on sushi restaurants near Midtown. The higher-ranked restaurants had stronger evidence about fresh food, stable quality, good service, and a comfortable atmosphere for two people.

For Test 2, we used ZIP code 10025, cuisine Korean, party size 4, and budget medium. The system gave more weight to group suitability. The Top-3 results matched the Korean cuisine requirement and included evidence about service, value, wait time, seating space, and overall dining experience.

For Test 3, we used ZIP code 10027, cuisine Italian, party size 6, and budget high. The system focused more on larger-group dining and higher-budget experience. The higher-

ranked restaurants showed stronger evidence of spacious seating, an upscale atmosphere, and group-friendly dining.

Overall, the test results showed that the system did not only rank restaurants by Google rating or distance. It combined user inputs, Google Places metadata, review-based RAG evidence, Gemini review analysis, and Gemini image analysis to produce more transparent recommendations.

6.3 Evaluation Criteria

We evaluated the system using three main criteria: relevance, groundedness, and interpretability.

Relevance means whether the recommended restaurants matched the user's input, including cuisine, budget, and party size. For example, a sushi query with a medium budget and party size of two should return sushi or Japanese restaurants with a reasonable price level and a suitable setting for two people.

Groundedness means whether the recommendation was supported by real evidence. In our system, each recommendation included review evidence from Google Places reviews and image evidence from restaurant photos. This helped make sure the recommendation was based on reviews and photos, not only generated text.

Interpretability means whether the user could understand why one restaurant was ranked higher than another. Our system showed a score breakdown for each Top-3 result, allowing users to inspect the main factors behind the ranking. This made the recommendation process more transparent.

Overall, these evaluation criteria helped us test both the technical pipeline and the quality of the final recommendations. The results showed whether the system could match user needs, support recommendations with evidence, and explain the ranking clearly.

7 Results and Analysis

7.1 Example Search Output

To evaluate the system output, I tested the website with a sushi-related dining query. As shown in Figure 3, the demo input was ZIP code 10025, cuisine preference Sushi, party size 2, and budget range Medium. After the user enters these preferences and clicks the "Find restaurants" button, the system sends the request to the backend search pipeline and returns a ranked list of restaurant recommendations. The project is designed to return transparent Top-3 recommendations with supporting evidence rather than only basic restaurant metadata.

DINECOMPASS · PERSONALIZED DINING GUIDE

Find restaurants that fit your plans.

DineCompass helps you discover nearby restaurants based on cuisine, budget, party size, and vibe — with recommendations supported by reviews, real photos, and clear reasoning.

Review-based insights

Evidence-backed picks

Photo-based vibe cues



Advanced settings ▲
API keys are stored locally in your browser.

Google Maps API Key
.....

Gemini API Key
.....

Where are you?
10025

What are you craving?
sushi

Number of people
2

Budget range
Medium

Find restaurants

The website returned three recommended restaurants: Hudson Sushi, Mizu Sushi, Ramen & Grill, and Catzuneko Shokudo | Sushi & Japanese Dining NYC. Each result card includes the restaurant label, rating, number of ratings, match score, address, summary, recommended dishes, reservation link, review-based evidence, VLM image analysis, and score breakdown. This output format allows users to compare restaurants not only by rating, but also by the reasons and evidence behind each recommendation.



BEST OVERALL
Catuzneko Shokudo | Sushi & Japanese Dining NYC

★ 4.9 259 reviews March 85

Why recommended: It's highly recommended for its fresh sushi, excellent service, and charming cat-themed ambiance.

518 9th Ave, New York, NY 10018, USA

Price level: Not available

Summary: Catuzneko Shokudo offers fresh, delicious sushi and sashimi, praised for excellent, friendly service and a fun, unique, cat-themed atmosphere, visually confirmed as casual and comfortable, making it suitable for small groups.

Recommended dishes: sashimi, spicy edamame, sushi, edamame

Reservation: Available ([Book now](#))

[View on Maps](#)





Review RAG Evidence

"I had a really great experience here. The sashimi was super fresh and cut into thick, generous pieces — you can really taste the quality. The spicy edamame was also delicious and full of flavor, definitely a must-order. The whole restaurant has a cat theme, which made the place extra fun and unique. There were cute cat details everywhere — on the cups, soy sauce dishes, wallpaper, and little decorations. It felt very thoughtful and charming. We left feeling full and very satisfied. The service was also excellent — everyone was friendly, welcoming, and attentive. Highly recommend this spot if you love good sushi and a cozy atmosphere!"

Shane Chen • In the last week • Rating 5

Matched terms: dishes, quality, service, sushi

"Great sushi restaurant. Loved the cat theme. Food was amazingly delicious. Service was courteous and fast. Loved it!"

Darryl Walker • In the last week • Rating 5

Matched terms: service, sushi

"Very friendly service and delicious food, I loved the cat theme"

C. Natalia Mateus • In the last week • Rating 5

Matched terms: service

VLM Image Analysis

Visual vibe: casual

Space impression: comfortable

Group suitability: good for small groups

Visual confidence: high

Food visual cues:

- elaborate sushi rolls
- colorful presentation
- prepared noodles

The images display a restaurant interior with Japanese-themed decor and various dishes, including elaborately prepared sushi rolls and a noodle dish.

► **Score Breakdown**

Using the top-ranked restaurant from this query, Hudson Sushi, as the main example (see Figure 4), the result card contains several components that explain both the recommendation and the evidence behind it. On the left side, the card displays restaurant photos, including the storefront and food images. These photos help users visually inspect the dining environment and food presentation before opening external links. On the top right side, the card shows the recommendation label “Best Overall,” the restaurant name, its Google rating, number of ratings, and match score. In this example, Hudson Sushi has a rating of 4.8, 1,154 ratings, and a match score of 77.

The “Why recommended” box provides a short explanation of why the restaurant was selected. For Hudson Sushi, the system recommends it for fresh sushi, excellent service, good value, and suitability for small groups. Below this explanation, the card shows the restaurant address, followed by a concise summary of the restaurant. The summary gives a more complete description of the restaurant’s food quality, service, value, and overall dining atmosphere.

The “Recommended dishes” field lists dishes or menu items extracted from the review analysis. For Hudson Sushi, the recommended items include sushi, Japanese soda, and rolls. The “Reservation” field shows whether booking information is available. When the card displays “Reservation: Available (Book now),” the “Book now” link can redirect users to the restaurant’s official booking page. The “View on Maps” button is also functional and redirects users to Google Maps, allowing them to check the restaurant location and navigate there directly.

The card also includes a “Review RAG Evidence” section. This section shows the selected review snippets used by the system as supporting evidence. For Hudson Sushi, the displayed reviews mention excellent service, fresh sushi, tasty food, and positive customer experiences. Each evidence box includes the review text, reviewer information, rating, and matched terms such as “service” or “sushi.” This makes the recommendation more grounded because users can see which review evidence contributed to the system’s reasoning.

The “VLM Image Analysis” section reports the photo-based observations for Hudson Sushi. In this example, the system describes the visual vibe as lively, group suitability as good for small groups, and visual confidence as medium. The image evidence summary notes that the restaurant entrance is heavily decorated and that outdoor seating is visible. These observations add concrete visual context to the recommendation card.

Finally, the “Score Breakdown” section explains how the match score is formed across the ranking components. For Hudson Sushi, the total score is 77.2. This breakdown improves interpretability because users can understand why the restaurant ranks highly instead of only seeing a final recommendation label.

7.2 Top-3 Recommendation Comparison Analysis

Table 1 compares the three restaurants returned by the system for the sushi query. The Top-3 results are Hudson Sushi, Mizu Sushi, Ramen & Grill, and Catzuneko Shokudo | Sushi & Japanese Dining NYC. The comparison shows that the system does not rank restaurants based only on Google ratings. Instead, it combines review evidence, visual image analysis, and an explainable score breakdown to support the final recommendation order.

Table 1. Comparison of the Top-3 Restaurant Recommendations

Restaurant	Label	Rating	Number of Ratings	Address	Recommended Dishes	Reservation
Hudson Sushi	Best Overall	4.80	1,154	4525 Broadway, Union City, NJ 07087, USA	sushi; Japanese soda; rolls	
Mizu Sushi, Ramen & Grill	Great Value	4.80	289	940 Amsterdam Ave, New York, NY 10025, USA	Hamachi Kama; hibachi; sushi platter; sushi	Available, with "Book now" link
Catzuneko Shokudo Sushi & Japanese Dining NYC	Hidden Gem	4.90	261	518 9th Ave, New York, NY 10018, USA	sashimi; spicy edamame; sushi; edamame	
Restaurant	External Link	Why Recommended	Review RAG Evidence Highlights	VLM Image Analysis Highlights	Total Score	
Hudson Sushi	"View on Maps" redirects to Google Maps	Fresh sushi, excellent service, good value, and suitable for small groups.	Reviews mention excellent service, fresh sushi, tasty food, and positive customer experiences. Matched terms include "service" and "sushi."	Visual vibe is lively. The system identifies it as good for small groups with medium visual confidence.	77.2 (Cuisine match: 10; Budget match: 20; Rating: 19.2; Review value match: 8; Vibe fit: 5; Visual vibe fit: 5; Evidence quality: 10; Wait penalty: 0)	
Mizu Sushi, Ramen & Grill		A casual sushi and hibachi restaurant with a broad menu and small-group suitability.	Reviews include positive comments on service, sushi, and hibachi.	Visual vibe is casual. The image analysis describes the space as spacious and suitable for small groups, with high visual confidence.	77.2 (Cuisine match: 10; Budget match: 20; Rating: 19.2; Review value match: 8; Vibe fit: 5; Visual vibe fit: 5; Evidence quality: 10; Wait penalty: 0)	
Catzuneko Shokudo Sushi & Japanese Dining NYC		Fresh sushi, good service, good value, and a unique cozy cat-themed atmosphere.	Reviews mention fresh sashimi, sushi, spicy edamame, friendly service, and cat-themed dining atmosphere.	Visual vibe is casual and cozy. The system describes the space as comfortable and suitable for small groups, with high visual confidence.	74.6 (Cuisine match: 10; Budget match: 20; Rating: 19.6; Review value match: 5; Vibe fit: 5; Visual vibe fit: 5; Evidence quality: 10; Wait penalty: 0)	

Hudson Sushi is ranked as "Best Overall" with a rating of 4.80, 1,154 ratings, and a total score of 77.2. Its recommendation is mainly supported by review evidence about fresh sushi, excellent service, good value, and suitability for small groups. The VLM image analysis also describes the visual vibe as lively and suitable for small groups. Its score breakdown is strong across the main components, including cuisine match 10, budget match 20, rating 19.2, review value match 8, vibe fit 5, visual vibe fit 5, evidence quality 10, and wait penalty 0.

Mizu Sushi, Ramen & Grill receives the same total score of 77.2, but its recommendation profile is slightly different. It is labeled as "Great Value" because it offers a broad sushi and hibachi menu and is also considered suitable for small groups. Its score breakdown is identical to Hudson Sushi, including cuisine match 10, budget match 20, rating 19.2, review value match 8, vibe fit 5, visual vibe fit 5, evidence quality 10, and wait penalty 0. Compared with Hudson Sushi, Mizu Sushi is presented more as a casual and value-oriented option, with a broader menu that includes both sushi and hibachi.

Catzuneko Shokudo is labeled as "Hidden Gem" and receives a slightly lower total score of 74.6. It has the highest rating among the three restaurants, with a rating of 4.90 and a rating component of 19.6. However, its review value match is lower at 5, compared with 8 for the first two restaurants. This explains why its total score is lower despite having the strongest Google rating. Its main strength is its distinctive cat-themed atmosphere, fresh sushi, and cozy visual vibe, which make it a more unique but slightly less broadly matched recommendation.

Overall, the Top-3 comparison shows that the ranking system balances multiple factors instead of relying only on Google rating. Hudson Sushi and Mizu Sushi score higher because they combine strong cuisine match, budget match, review value, visual suitability, and evidence quality. Catzuneko Shokudo performs especially well in rating and atmosphere, but its lower review value match leads to a slightly lower final score. This comparison demonstrates how the score breakdown makes the ranking more interpretable and helps explain why each restaurant appears in a different position.

7.3 Comparative Evaluation

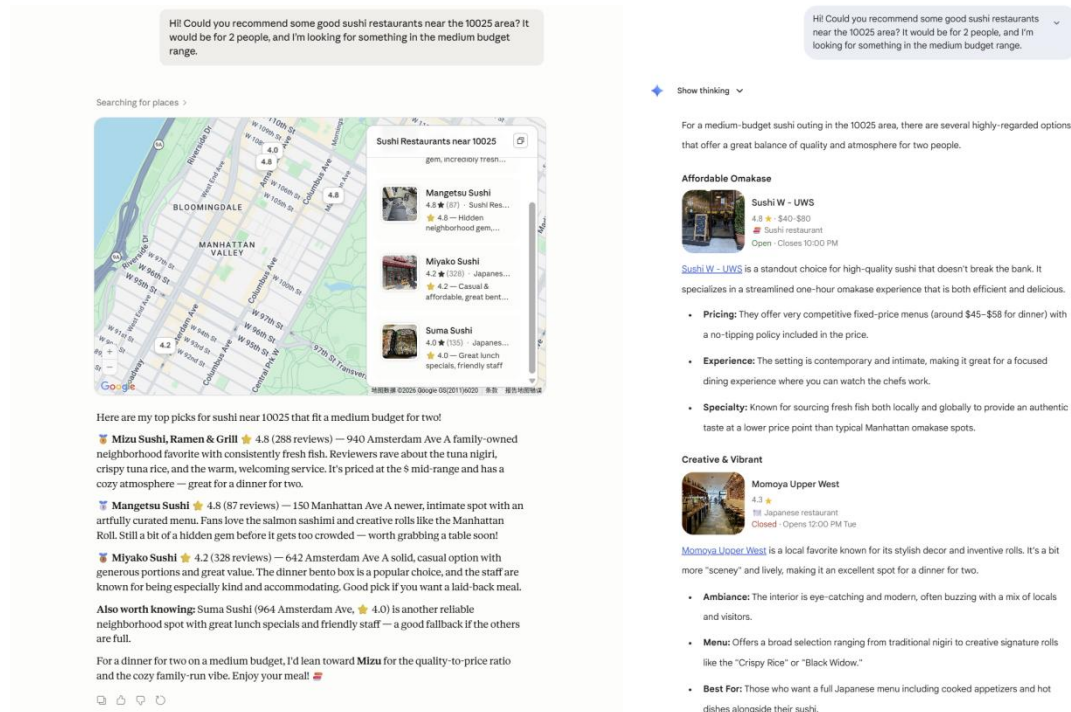


Figure 5. LLM-Only Recommendation Outputs from Claude and Gemini

We also compared DineCompass with general LLM outputs from Claude and Gemini using the same sushi search request. Both models can generate reasonable sushi recommendations for the same user request, but their outputs are mainly text-based and less structured. They provide restaurant names and short explanations, but they do not consistently show the full recommendation pipeline behind the answer, such as retrieved review evidence, VLM-based image analysis, score breakdown, reservation links, and map navigation in one organized interface. In contrast, our platform presents the results in a more visually clear and actionable format. Each recommendation card combines photos, rating, number of ratings, match score, summary, recommended dishes, review evidence, image-based vibe analysis, and external links. This makes our output easier to inspect, compare, and use for real dining decisions.

Compared with restaurant search platforms such as Google Maps and Yelp, our system is more focused on personalized reasoning and explanation. Google Maps and Yelp are strong general-purpose discovery tools because they provide business listings, ratings, reviews, photos, maps, and search filters. Google Maps also supports reviews, ratings, and user-

uploaded photos or videos, while Yelp allows users to search by keywords and apply filters to narrow results. However, these platforms usually require users to manually read reviews, inspect photos, compare options, and infer which restaurant best matches their specific request. Our system reduces this manual effort by combining review-based retrieval, LLM summary, VLM visual interpretation, and rule-based ranking into a single recommendation card. Therefore, the main advantage of our platform is not replacing Google Maps or Yelp, but turning scattered restaurant information into an evidence-backed, visually organized, and explainable recommendation experience.

7.4 User Studies

To further evaluate the practical usefulness of DineCompass, we shared the completed web application with classmates and asked them to test the system using their own restaurant search preferences. After using the website, participants completed a Google Forms feedback survey. The survey responses were exported into an Excel file and analyzed to understand users' perceptions of usability, usefulness, evidence quality, and comparison with existing restaurant search tools.

In total, we collected 145 user responses. Overall, the feedback suggests that users found DineCompass useful and practical for restaurant decision-making. The strongest results were related to trust, explainability, and structured comparison. For example, the statement "The review evidence made the AI recommendation more trustworthy" received an average score of 4.06 out of 5, and 74.5% of users selected either 4 or 5. Similarly, the score breakdown received an average score of 4.05, showing that users appreciated being able to understand why one restaurant was ranked higher than another. The overall usefulness of the web app also received a positive average score of 3.95, with 71.7% of responses rating it 4 or 5.

The survey also shows that users preferred DineCompass over other restaurant search options. When asked which tool they would prefer to use for finding restaurants, 107 out of 145 users, or 73.8%, selected our AI restaurant recommendation web app. This suggests that users valued the combination of Top-3 ranking, AI-generated summaries, review evidence, image analysis, and score breakdown. In the open-ended responses, users often described the app as easy to use, clear, practical, and helpful for saving time. Several users also mentioned that the review evidence and score breakdown made the recommendations easier to trust and understand. At the same time, some users suggested future improvements, such as adding more filters, improving image analysis, supporting saved preferences, and using semantic search for review retrieval.

Overall, the user study supports the main goal of DineCompass. The results indicate that users did not only value the final restaurant recommendation, but also valued the evidence and explanation behind the recommendation. This suggests that an evidence-grounded recommendation interface can make AI restaurant suggestions more transparent, trustworthy, and useful for real dining decisions.

Table 2. User Feedback Survey Results for DineCompass

Survey Item	Average Score/Result	Positive Response Rate
Ease of Use	3.79/5	71.0%
AI Summary Helpfulness	3.92/5	72.4%
Review Evidence Trust	4.06/5	74.5%
Image Analysis Helpfulness	3.90/5	69.0%
Score Breakdown Clarity	4.05/5	73.8%
Overall Usefulness	3.95/5	71.7%
Clearer than Google Maps	3.70/5	62.1%
Save Time vs. Reading Reviews	3.87/5	66.9%
More Grounded than General AI	3.87/5	69.0%
Preferred DineCompass	107/145	73.8%

7.5 Error Analysis and Limitations

Although the system provides interpretable and evidence-based recommendations, the output still has several limitations. First, the review evidence is limited by the number and quality of Google reviews returned by the API, so the selected review snippets may not fully represent all customer opinions. Second, restaurant photos may not perfectly reflect the real dining experience because they can be outdated, staged, or focused on only a small part of the restaurant. Third, the LLM and VLM outputs depend heavily on the quality of the retrieved reviews and images, so weak input evidence can lead to less informative summaries. Finally, because the system relies on external APIs such as Google Places and Gemini, API quota limits, network issues, or missing fields can affect the stability and completeness of the results.

8 Discussion

8.1 Implications

This project shows that large language models can be used for more than simple chatbot conversations. In our restaurant recommendation system, the LLM works as part of a larger pipeline. It helps read restaurant reviews, summarize useful information, and explain why a restaurant fits the user’s needs.

Our system does not only return a list of restaurants. It gives Top-3 recommendations with supporting evidence. This makes the results more useful because users can see why each restaurant was recommended, instead of only seeing a ranking.

The project also shows the value of combining RAG, VLM, and rule-based ranking. The RAG component helps the system use real review evidence. The VLM component adds visual context from restaurant photos, such as atmosphere, food presentation, seating space, and group suitability. The rule-based ranking component makes the final score easier to explain because it is based on transparent scoring factors rather than a black-box decision.

Overall, this project shows that an LLM-based recommendation system works better when it is combined with structured retrieval and transparent ranking rules. The LLM does not replace the whole recommender system. Instead, it helps turn messy review and photo information into clear, human-readable insights. This is useful for restaurant recommendation because users often care not only about which restaurant is recommended, but also why it is recommended.

8.2 Limitations

Although our system shows a useful end-to-end restaurant recommendation pipeline, it still has several limitations.

First, the system does not use real user data, such as clicks, bookings, or satisfaction feedback. Because of this, we cannot fully know whether users would actually choose or like the recommended restaurants in real life. Our evaluation mainly checks whether the results are relevant, evidence-based, and easy to understand.

Second, we did not compare our system with a simple baseline recommender. For example, we did not compare it with rankings based only on Google rating, distance, price level, or keyword match. Because of this, we cannot clearly measure how much improvement comes from using LLM, RAG, and VLM.

Third, the system depends on external APIs, especially the Google Maps / Google Places API and the Gemini API. If restaurant reviews, photos, ratings, or metadata are missing or outdated, the final recommendation may be less accurate. Also, if Gemini gives inconsistent review summaries or image analysis, the final explanation may be affected.

Fourth, the current ranking weights are manually designed. Although the score breakdown is easy to understand, the weights are not learned from real user behavior. Different users may care about different things, such as food quality, price, service, quietness, or group suitability.

Finally, privacy and security need more attention. The system requires users to enter API keys and ZIP codes. ZIP codes can still show approximate user location, and API keys must be protected from exposure or misuse. In a production version, API keys should be managed safely on the backend instead of being entered directly through the frontend.

8.3 Future Work

Future work could improve the system in several ways. First, we could add user history and personal preferences. For example, if a user often chooses quiet restaurants or places with good service, the system could use this information to make better recommendations.

Second, we could improve the review retrieval step by using embedding-based semantic retrieval instead of simple keyword matching. This would help the system find reviews with similar meanings, even when they do not use the exact same words.

Third, we could add more evaluation metrics, such as NDCG, MRR, and a user satisfaction survey. This would make the evaluation more formal and easier to compare with other recommendation methods.

Fourth, we could improve the ranking module by adjusting the ranking weights or training a learning-to-rank model. This would make the final ranking more data-driven instead of only using manually designed weights.

Finally, we could support more dining scenarios, such as date night, group dinner, quiet study cafe, birthday dinner, business lunch, or casual hangout. Different scenarios may need different ranking logic, so this would make the system more flexible and closer to real user needs.

9 Conclusion

This project developed a runnable personalized restaurant recommendation web application. The system helps users find restaurants based on location, cuisine preference, party size, budget, and dining needs. Instead of only ranking restaurants by rating or distance, it combines restaurant metadata, review evidence, image analysis, and rule-based ranking to generate more useful and explainable recommendations.

The system integrates restaurant metadata, review-based evidence, image-based observations, and interpretable ranking into one recommendation interface. This design allows users to compare restaurants based not only on ratings, but also on the evidence and reasoning behind each recommendation.

Overall, this project shows that LLMs can be useful inside a recommendation system as a reasoning and summarization layer. By grounding recommendations in review evidence, adding visual information from photos, and using an explainable ranking strategy, the system produces restaurant recommendations that are closer to real dining decisions. The final application demonstrates a practical end-to-end approach for building transparent, evidence-based, and personalized restaurant recommendations.

References

- [1] Bao, J., Zheng, Y., & Mokbel, M. F. (2012). Location-based and preference-aware recommendation using sparse geo-social networking data. *Proceedings of the 20th International Conference on Advances in Geographic Information Systems*, 199–208.
- [2] Pontiki, M., Galanis, D., Pavlopoulos, J., Papageorgiou, H., Androutsopoulos, I., & Manandhar, S. (2014). SemEval-2014 Task 4: Aspect based sentiment analysis. *Proceedings of the 8th International Workshop on Semantic Evaluation (SemEval 2014)*.
- [3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://arxiv.org/abs/2005.11401>
- [4] Zhang, Y., & Chen, X. (2020). Explainable recommendation: A survey and new perspectives. *Foundations and Trends® in Information Retrieval*, 14(1), 1–101.
- [5] Yilmaz, G. B. (2025). *Best-restaurants-route-planner-agent* [Computer software]. GitHub. <https://github.com/gulbike/Best-Restaurants-Route-Planner-Agent>
- [6] Buluslan. (2026). *Ecommerce-competitor-analyzer* [AI agent skill]. GitHub. <https://github.com/buluslan/ecommerce-competitor-analyzer>
- [7] Google. (2026). *Places API documentation*. Google Maps Platform. <https://developers.google.com/maps/documentation/places/web-service/overview>
- [8] Google. (2026). *Gemini API documentation*. Google AI for Developers. <https://ai.google.dev/gemini-api/docs>
- [9] Li, J., Li, D., Xiong, C., & Hoi, S. C. H. (2022). BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. *Proceedings of the 39th International Conference on Machine Learning*. <https://arxiv.org/abs/2201.12086>
- [10] Wu, L., Zheng, Z., Qiu, Z., Wang, H., Gu, H., Shen, T., Qin, C., Zhu, C., Zhu, H., Liu, Q., Xiong, H., & Chen, E. (2023). A survey on large language models for recommendation. *arXiv*. <https://arxiv.org/abs/2305.19860>